

Experiment Design for Benchmarking Planning Architectures for Single-UAV Autonomy

Concise working plan

Research question: Under the same UAV tasks, observations, controller, and safety constraints, which planning architecture provides the best trade-off between success, safety, recovery, latency, and efficiency?

1. Experimental Scope

- Architecture-level benchmark, not a new planner.
- Keep perception, controller, safety limits, observations, and task conditions identical.
- Use three task families and five core architectures.

2. Tasks and Benchmarks

Task family	Benchmark	Purpose
Instruction-following navigation	AerialVLN	Long-horizon language-guided navigation.
Semantic object/goal search	UAV-ON or OpenUAV / UAV-Need-Help	Open-vocabulary search and grounding.
Reasoning and recovery	CognitiveDroneBench + failure variants	Ambiguity, blocked routes, wrong targets, and delay.

3. Architectures to Compare

Architecture	Role
Classical baseline	Classical planner and controller only.
LLM tool planner	Mission decomposition and selection of validated skills/API calls.
VLM semantic waypointer	Visual target, direction, or waypoint selection.
Hybrid stack	LLM → VLM/waypoint → safety gate → classical execution.
Selective recovery supervisor	LLM invoked only after failure, ambiguity, or no progress.
Optional direct VLA	Pretrained VLA local-action policy, only if integration is practical.

4. Simulator and Models

- Main environment: AirSim/Unreal. Secondary validation: PX4 + Gazebo for dynamics and recovery.
- Use one common planner input/output interface across all architectures.
- Run one reproducible local/open model and one strong API model as an upper-bound reference.
- Use one or two models per modality; avoid a large model leaderboard.

5. Fair Comparison Protocol

- Freeze prompts, tools, safety rules, action frequency, and observation format before final runs.
- Use the same scenarios, several random seeds, and 20–30 repetitions per condition.
- Inject blocked routes, moved/hidden targets, ambiguity, sensor noise, stale images, drift, and API delay.
- Log proposals, safety rejections, calls, recovery attempts, and outcomes.
- Core size: 3 tasks × 5 architectures × 2 model conditions × 20–30 episodes.

6. Evaluation Metrics

Primary metrics should support the main conclusions; all others are diagnostic.

Category	Primary metric	Definition
Performance	Task Success Rate	Percentage of missions completed.
Safety	Safety Violation Rate	Collisions, near-collisions, invalid actions, or geofence violations.
Robustness	Recovery Success Rate	Percentage of failure cases recovered.
Real-time	Planning Latency	Time needed to produce the next decision.
Efficiency	SPL/path efficiency or model cost	Navigation efficiency or calls/tokens/API cost.

Diagnostic Metrics

- Executable Plan Rate — whether proposed commands can be executed.
- Unsafe Proposal Rate — how often the safety layer rejects output.
- Stale Action Rate — how often latency makes an action outdated.
- False-Target Commitment — how often the wrong target is followed.
- Recovery Time and Human Interventions — used for failure analysis.

7. Drawing Conclusions

- Do not use one overall score unless the weighting is formally justified.
- Report trade-offs: one architecture may be more successful, another safer, and another faster or cheaper.
- Use diagnostic metrics only to explain failures and successes.
- Hypothesis: hybrid/selective architectures offer the best safety–recovery trade-off; direct VLM/VLA may be stronger for short-horizon visual behavior.

8. Recommended Work Order

1. Reproduce one AerialVLN baseline.
2. Define the common planner interface.
3. Implement classical, LLM, VLM, hybrid, and selective-recovery baselines.
4. Add semantic-search and controlled-failure scenarios.
5. Freeze prompts/configurations and run repeated experiments.
6. Perform statistical analysis; add direct VLA only if practical.

Q1: Which architectural components produce the best autonomy trade-offs under fixed UAV tasks and hardware constraints?

או אז שלב ב

RQ2: Can an autonomous UAV dynamically select its own reasoning architecture during a mission based on task state, uncertainty and compute budget?

Ablation per task ? what is enough..